The Syndrome Buffer Sizing Problem in Quantum Error Correction

Sumeet Shirgure[1, *] and Siyuan Niu[1, †]

[1] *Department of Electrical and Computer Engineering, University of Central Florida*

(Dated: October 14, 2026)

A fault-tolerant quantum computer must continuously decode a stream of syndrome data, yet the computational work required by each decoding window is random. Although average throughput determines whether the syndrome backlog is stable (doesn't grow without bound), it does not determine the buffer capacity needed to meet a target overflow probability. Here, we model syndrome buffering as a queue and show how its required depth depends on the tail of the decoding-work distribution. For heavy-tailed work, established queueing results imply that sliding-window decoding can require buffer depths that grow superpolynomially in the logarithm of the inverse overflow probability. Parallel-window decoding suppresses the overflow tail through concurrent service, with the suppression exponent determined by the number of spare decoders; under out-of-order release, the number of spare decoders required for a fixed buffer and target overflow probability grows logarithmically with the inverse target probability. We find evidence of heavy-tailed decoding work in Relay-BP on the gross-code and minimum-weight perfect matching on the surface code. These results identify syndrome-buffer capacity and memory-release policy as important design considerations for real-time quantum error correction.

I. INTRODUCTION

Quantum error correction (QEC) turns a noisy quantum processor into a reliable one only if the classical decoder keeps up with the syndrome data the processor produces [1, 2]. Each QEC cycle emits a fixed number of syndrome bits, so the input to the decoder is a deterministic stream. If the decoder processes data more slowly than it arrives, unprocessed syndromes accumulate without bound; because logical non-Clifford gates must wait for decoded outcomes, the computation slows exponentially in its depth. This is the *backlog problem* [1, 3]. Sliding-window decoding (SWD) [4] and parallel-window decoding (PWD) [5, 6] address it by cutting the stream into windows and, in the parallel case, decoding several windows at once.

Avoiding a backlog is a statement about *throughput*. It does not say what happens when an individual window takes much longer than average to decode. The leading decoders for the surface code: minimum-weight perfect matching (MWPM) [7] and union-find (UF) [8], and for high-rate quantum low-density parity-check (qLDPC) codes belief propagation (BP) and its variants, such as Relay-BP [9], all have data-dependent run times. For a fixed decoding window, the work S is a random variable whose distribution depends on the code, the decoder and the noise.

We restrict attention to a quantum-memory experiment, a code block idling under circuit-level noise, which is the setting in which current real-time decoders are benchmarked [10, 11]. We make three contributions. First, we show that in this setting the backlog problem is exactly the stability problem of the syndrome buffer, viewed as a queue; once the queue is stable, the remaining question is how deep the buffer must be. (During a logical computation, feed-forward makes the arrival of windows depend on decoding outcomes, and the equivalence need not be exact.) Second, we collect asymptotic results from queueing theory showing that this depth is logarithmic in the target failure probability for light-tailed work but grows superpolynomially in its logarithm for heavy-tailed work, and we identify three ways to provision for this. Third, we show statistically that the work of Relay-BP on the gross code and of MWPM on the surface code is heavy-tailed.

II. VARIABLE-TIME DECODERS

A decoder provisioned for its worst case must accept either a long fixed latency or a simplified, less accurate algorithm. A decoder provisioned for its average case can run an accurate algorithm with short typical latency, as long as its mean work stays below the syndrome arrival period, that is,

$$\rho \;=\; \frac{\mathbb{E}[S]}{a} \;<\; 1, \tag{1}$$

where a is the work budget available per committed window (the commit period expressed in the same units as S) and ρ is the load. For superconducting qubits, with a syndrome cycle of about $1\,\mu$s [10], the budget is a latency rate of $1\,\mu$s of decoding per syndrome cycle: a window that commits d cycles must, on average, be decoded in $d\,\mu$s, and a is the number of operations the decoding hardware completes in that time.

Work versus latency. Throughout, S is an algorithmic operation count: the number of bounded-cost operations a decoder performs on one window. It models the amount of work done, not wall-clock time. We assume that the latency of a single decode (its service time T) equals this

* sumeetparameshwar.shirgure@ucf.edu
† siyuan.niu@ucf.edu

work up to scaling, $T \approx S/\kappa$, where κ is the effective operation throughput of the decoding hardware; hardware implementations can parallelize a single decode, which raises κ but does not change the shape of the distribution. More generally it suffices that $c_1 S \le T \le c_2 S$ for constants $0 < c_1 \le c_2$. Such a rescaling preserves the tail class and tail exponents of S, and so changes only the constants in the buffer depths below, not their asymptotic growth.

Capping versus completion. A real-time system can deal with the slow tail of S in two ways: it can *cap* the decoder at a work budget, or it can *buffer* the syndrome stream so that a slow decode delays later windows instead of failing. Which option is available depends on the decoder. MWPM and UF must run to completion: a partial matching or a partially grown cluster forest does not yield a correction consistent with the syndrome, so capping them means aborting the decode and either declaring a logical failure or invoking a fallback decoder. BP-family decoders can instead be truncated, since the number of iterations (for Relay-BP, of relay legs) is a free parameter. Either way, a cap adds a failure channel and raises the logical error rate (LER).

The rare-event regime. When the decoder is not capped, or is capped far above its median, slow decodes are absorbed by the buffer. Our analysis concerns this regime and makes one assumption explicit: the rare decodes that determine the buffer depth at the target failure rate take much longer than the syndrome-extraction time scale, $S \gg a$. The queueing results below are leading-order asymptotics in this limit, and all our conclusions are subject to this assumption. If the typical duration of rare decodes is only a few commit periods, a short buffer absorbs them, and our estimates and conclusions may be an overestimate.

III. THE BACKLOG PROBLEM IS THE STABILITY PROBLEM OF THE BUFFER

Buffering the syndrome stream turns the decoder into a queue [12–14]. In Kendall's notation [15], a D/G/k queue has deterministic interarrival times (D), independent and identically distributed service times with a general distribution (G), and k servers fed from a single queue; D/G/1 is its single-server case. Windows arrive deterministically, one per commit period, and window n requires work S_n. In SWD a window cannot be committed until its predecessor has been, so there is a single server and the system is a D/G/1 queue. Let W_n be the backlog of unfinished work, in work units, that window n finds when it arrives (its waiting time). It obeys Lindley's recursion [16]

$$W_{n+1} \;=\; \max\bigl(0,\; W_n + S_n - a\bigr), \qquad W_0 = 0. \tag{2}$$

Unrolling Eq. (2) and reversing the order of the i.i.d. increments gives $W_n \overset{d}{=} \max_{0\le j\le n} \sum_{i=1}^{j} (S_i - a)$. Its limit in distribution,

$$W \;=\; \lim_{n\to\infty} W_n \;\overset{d}{=}\; \sup_{j\ge 0} \sum_{i=1}^{j} (S_i - a), \tag{3}$$

is the stationary backlog. By Loynes' theorem [17], W is finite almost surely exactly when $\rho < 1$; for $\rho \ge 1$ the backlog W_n grows without bound. That is Terhal's backlog problem. In a memory experiment, avoiding the backlog problem and making the buffer stable are therefore the same requirement, Eq. (1). Every W below is this stationary limit.

The connection goes further than the stability condition. Terhal's argument for the backlog is a geometric series [1]: syndrome data that arrive while the decoder works through a backlog must themselves be decoded, which takes a fraction ρ as long, during which further data arrive, and so on. The total delay scales as $1+\rho+\rho^2+\cdots$, which converges to $1/(1-\rho)$ for $\rho < 1$ and diverges otherwise. The same series appears in Kingman's formula for the mean stationary backlog of a single-server queue [18],

$$\mathbb{E}[W] \;\approx\; \frac{\rho}{1-\rho} \cdot \frac{c_a^2 + c_s^2}{2} \cdot \mathbb{E}[S], \tag{4}$$

where c_a and c_s are the coefficients of variation (standard deviation divided by mean) of the interarrival times and of S. Equation (4) becomes exact as $\rho \to 1$. Syndrome windows arrive deterministically, so $c_a = 0$, and the right-hand side becomes $\mathrm{Var}(S)/[2(a - \mathbb{E}[S])]$, which for the D/G/1 queue is also Kingman's upper bound on $\mathbb{E}[W]$ at every $\rho < 1$. This is Terhal's divergence at $\rho \to 1$, scaled by how variable the decoding work is. It requires $\mathrm{Var}(S) < \infty$, which, as we show below, some decoders violate.

How a stable buffer behaves. When $\rho < 1$ the decoder drains the buffer faster than it fills on average, so the buffer is nearly empty most of the time. A single slow window stops the drain while the inflow continues, so the backlog rises in proportion to that window's work, and then drains only at the spare rate $a(1-\rho)$ per window. The occupancy is a sequence of sharp spikes with slow decays, and overflow is decided by the tail of S, not its mean.

Buffer occupancy is set by the sojourn time. The backlog W is measured in work, not in stored syndrome bits. The buffer holds the syndrome data of every window that has arrived but has not yet been committed, including the window being decoded. Window n arrives at time na (time in work units, with the decoder completing one unit of work per unit time) and is committed after its *sojourn time*

$$V_n \;=\; W_n + S_n, \tag{5}$$

the time it waits plus the time it is decoded, so it occupies the buffer during $[na,\, na + V_n)$. Let N_n be the number of windows in the buffer just after window n arrives, and let β be the number of syndrome bits per committed window,

so that a buffer of B windows holds βB bits. In SWD windows are committed in arrival order, so if window $n-B$ is still in the buffer, so are windows $n-B+1,\dots,n$. Hence $N_n > B$ exactly when window $n-B$ is not yet committed at time na, that is, when $V_{n-B} > Ba$. In the stationary limit the buffer overflows on arrival with probability

$$P(N > B) \;=\; P(V > Ba) \;=\; P\big(W > (B-1)a\big), \quad (6)$$

where V is the stationary sojourn time. The second equality follows from Lindley's recursion, which reads $W_{n+1} = \max(0, V_n - a)$. A buffer of B windows therefore tolerates a backlog of $B-1$ windows' worth of work, the remaining slot holding the window in service. (The partially written window currently arriving needs one more slot, which we absorb into B.)

We treat a buffer overflow as a failure to decode the affected window. The total failure budget per window is then

$$P_L + P(V > Ba) \;\le\; \varepsilon, \quad (7)$$

where B is the buffer depth in windows (it holds βB syndrome bits) and ε is the target failure probability. We size the buffer so that $P(V > Ba) \approx \varepsilon$, and use $\varepsilon = 10^{-12}$.

PWD [5, 6] decouples neighbouring windows so that up to k decoders work at once. We approximate it by a D/G/k queue with load $\rho = \mathbb{E}[S]/a < k$, in which windows start decoding in arrival order but, with out-of-order release, leave the buffer as each one commits. Departures are then no longer ordered, so the occupancy is not governed by a single sojourn time. The waiting windows, however, are still the most recent arrivals, so $N_n > B \ge k$ exactly when window $n-(B-k)$ has not started decoding at time na, and $P(N > B) = P(W > (B-k)a)$, where W is now the stationary waiting time of the D/G/k queue. For $k = 1$ this is Eq. (6). This approximation is optimistic in three ways. It ignores the dependency of each second-layer window on its neighbouring first-layer windows; it ignores correlations between overlapping windows; and it assumes that buffer memory is released *out of order*, as each window commits. With an in-order (first-in, first-out) ring buffer, a single stuck window pins all memory behind it and the system behaves like a single server regardless of k. The D/G/k results below are therefore lower bounds on the buffer depth that PWD requires.

IV. BUFFER DEPTH IS SET BY THE TAIL CLASS OF THE DECODING WORK

Define $\ell = \ln(1/\varepsilon)$, the protection level in natural units. The required depth $B(\varepsilon)$, in windows, is the smallest B with $P(N > B) \le \varepsilon$. By Eq. (6) and its D/G/k analogue, $B(\varepsilon) = k + x(\varepsilon)/a$, where $x(\varepsilon)$ solves $P(W > x) = \varepsilon$; we quote depths in work units, $x(\varepsilon) \approx B(\varepsilon)\, a$, since the offset of k windows does not affect the asymptotics. The buffer then holds βB bits. Its growth with ℓ depends almost entirely on the tail of S and only weakly on the load. Table I collects the leading-order results for three tail classes.

Light tails. If S has an exponential moment, the stationary D/G/1 backlog has an exponential tail, $P(W > x) \sim Ce^{-\gamma x}$, where $\gamma > 0$ is the Cramér–Lundberg exponent [14], the root of $\mathbb{E}[e^{\gamma(S-a)}] = 1$. By Eq. (6) the sojourn time then has the same exponent, $P(V > x) = P(W > x - a) \sim Ce^{\gamma a}e^{-\gamma x}$. The depth then grows linearly in ℓ, that is, logarithmically in $1/\varepsilon$. The same holds for multi-server queues [19], with a different exponent γ_k.

Heavy tails. Let $\bar{F}(x) = P(S > x)$ be the survival function of the work per window, and define its *integrated tail*

$$\bar{F}_I(x) \;=\; \frac{1}{\mathbb{E}[S]} \int_x^\infty \bar{F}(y)\, dy. \quad (8)$$

$\bar{F}_I$ is itself a survival function: that of the *remaining* work of the decode in progress, seen at a random instant. It is always heavier than $\bar{F}$; a power law $\bar{F}(x) \propto x^{-\alpha}$ gives $\bar{F}_I(x) \propto x^{-(\alpha-1)}$. If $\bar{F}_I$ is subexponential [20], a large backlog is almost always caused by a single huge decode rather than by gradual accumulation, and the stationary D/G/1 backlog satisfies [21–23]

$$P(W > x) \;\sim\; \frac{\rho}{1-\rho}\, \bar{F}_I(x), \quad (9)$$

where $f(x) \sim g(x)$ means $f(x)/g(x) \to 1$ as $x \to \infty$. A subexponential $\bar{F}_I$ is long-tailed, $\bar{F}_I(x-a) \sim \bar{F}_I(x)$, so by Eq. (6) the sojourn time has the same leading-order tail, $P(V > x) \sim \frac{\rho}{1-\rho}\bar{F}_I(x)$. The same follows from Eq. (5) directly: S_n is independent of W_n and its tail is lighter, $\bar{F}(x) = o(\bar{F}_I(x))$, so the decode of the window itself adds only a lower-order term. The overflow probability of a buffer of B windows is therefore $\frac{\rho}{1-\rho}\bar{F}_I(Ba)$ to leading order. The load enters only through the prefactor, so overprovisioning throughput does little. For k servers, Foss and Korshunov showed that large delays require $m = k - \lfloor \rho \rfloor$ simultaneous large service times, and that $P(W > x)$ is of the order of $\bar{F}_I(x)^m$ (with x rescaled by constants depending on ρ) [24]. We call m the number of *spare servers*. Intuitively, at any moment about ρ of the k decoders are busy on average, so about $k - \rho$ of them are free; for non-integer ρ, $m = \lceil k - \rho \rceil$ is this average number of free servers, rounded up. A single slow window ties up only one decoder, and the free ones absorb it. Overflow requires all m free decoders to be stuck on slow windows at the same time. Because each slow decode is rare, these probabilities multiply, and the tail probability enters raised to the power m. Substituting the lognormal and Pareto tails into these expressions gives the entries in Table I.

The classification has a simple consequence. A light-tailed decoder needs a buffer that grows linearly in the number of nines of protection. Lognormal and power-law tails make it superpolynomial: $e^{\sigma\sqrt{2\ell}}$ for the lognormal and $e^{\ell/(\alpha-1)}$ for the power law. The lognormal is asymptotically milder, subexponential in ℓ rather than exponential, but neither is polynomial in ℓ. Since the target ε

TABLE I. Leading-order growth of the buffer depth with the protection level. Entries give $x(\varepsilon) \approx B(\varepsilon)\, a$, the depth in work units, where $B(\varepsilon)$ is the smallest number of buffered windows with overflow probability $P(N > B) \le \varepsilon$ [for D/G/1, $P(V > Ba)$, Eq. (6)]; the buffer holds βB syndrome bits. $\ell = \ln(1/\varepsilon)$. Constant prefactors and factors of order $e^{O(1)}$ are omitted.

Tail class of S	$P(S > x)$	SWD, D/G/1	PWD, D/G/k	Growth in ℓ	Refs.
Light (exponential moment)	$\le Ce^{-\theta x}$	ℓ/γ	ℓ/γ_k	linear	[14, 19]
Lognormal (μ, σ)	$\bar{\Phi}\big(\frac{\ln x - \mu}{\sigma}\big)$	$e^{\mu+\sigma\sqrt{2\ell}}$	$e^{\mu+\sigma\sqrt{2\ell/m}}$	superpolynomial	[21–24]
Power law (Pareto, $\alpha > 1$)	$(x/x_0)^{-\alpha}$	$\propto e^{\ell/(\alpha-1)}$	$\propto e^{\ell/(m(\alpha-1))}$	exponential	[21, 22, 24]

S: decoding work per window; a: work budget per committed window; $\rho = \mathbb{E}[S]/a$: load ($\rho < 1$ for D/G/1, $\rho < k$ for D/G/k); $V = W + S$: sojourn time of a window (waiting plus decoding); β: syndrome bits per committed window; k: number of decoders; $m = k - \lfloor\rho\rfloor$: spare servers (the average number of free decoders, rounded up); γ: root of $\mathbb{E}[e^{\gamma(S-a)}] = 1$; γ_k: logarithmic decay rate of the k-server backlog; θ: decay rate of the light tail; $\bar{\Phi}$: standard normal survival function; μ, σ: parameters of $\ln S$; x_0, α: Pareto scale and index.

for a useful algorithm shrinks as the algorithm grows, ℓ grows with problem size, and so does the gap between a light-tailed and a heavy-tailed decoder.

For PWD with out-of-order release, the spare servers enter the exponent: the lognormal depth falls from $e^{\sigma\sqrt{2\ell}}$ to $e^{\sigma\sqrt{2\ell/m}}$, and the power-law exponent is divided by m. Parallel windows change the constant for a light tail, but for a heavy tail they change the growth rate.

Three ways to provision. For the power-law tails that our measurements support (below), SWD needs $B \propto \varepsilon^{-1/(\alpha-1)} = e^{\ell/(\alpha-1)}$, a buffer exponential in $\log(1/\varepsilon)$. If $\varepsilon \sim 1/N$, where N is the circuit size or computation volume, this is polynomial in N, $B \propto N^{1/(\alpha-1)}$, so the buffer remains efficient in the complexity-theoretic sense. It may still be unacceptable in practice. The alternative is PWD with $k = \lfloor\rho\rfloor + m$ decoders, so that the shock from a rare slow decode is absorbed by the spare decoders. Keeping the buffer at a fixed multiple b of the typical work requires $\varepsilon^{-1/(m(\alpha-1))} \lesssim b$, that is,

$$m \gtrsim \frac{\ell}{(\alpha-1)\ln b} = O\left(\log\frac{1}{\varepsilon}\right), \tag{10}$$

and the lognormal case likewise gives $m \gtrsim 2\sigma^2\ell/(\ln b - \mu)^2$. We therefore see three reasonable options for a heavy-tailed decoder. (i) Cap the decoder latency and pay in LER; for MWPM and UF this means aborting, and for BP-family decoders it means truncating. (ii) Use SWD with a very long buffer, which can be a simple first-in, first-out structure but whose size is exponential in ℓ and which is idle almost all of the time. (iii) Use PWD with out-of-order release and $m+\rho$ decoders, with m of order $\log(1/\varepsilon)$ from Eq. (10). Although the parallel-window construction of Ref. [5] extends formally to decoding hypergraphs, it has been demonstrated only with matching and union-find inner decoders, and we are not aware of any demonstration of parallel-window decoding with a BP-family decoder.

V. DECODING WORK IS HEAVY-TAILED

We characterised the tail of S for two code–decoder combinations under uniform circuit-level depolarizing noise, with 10^6 fully decoded d-round windows per point. For the gross code $[[144, 12, 12]]$ [25] we used Relay-BP [9] at $p \in \{10^{-3}, 3\times10^{-3}\}$. For the rotated surface code at $d \in \{11, 15\}$ and the same two values of p we used MWPM [7] and a peeling UF decoder [8, 26]. Figure 1 shows the empirical survival functions; details of the decoders, work metrics and statistical tests are in the Methods.

Peaks over threshold. We estimate a single tail parameter. By the Pickands–Balkema–de Haan theorem [27, 28], for a broad class of distributions the excess $Y = S - u$ over a high threshold u, conditioned on $S > u$, is approximately distributed as a generalized Pareto distribution (GPD),

$$P(Y > y \mid S > u) \approx \left(1 + \frac{\xi y}{\sigma}\right)^{-1/\xi}, \tag{11}$$

with scale $\sigma > 0$ and shape ξ. The shape interpolates continuously between the tail classes of Table I. As $\xi \to 0$, Eq. (11) becomes $e^{-y/\sigma}$, an exponential tail. For $\xi > 0$ it is a power law, $P(S > x) \sim x^{-1/\xi}$, which is Pareto-like with index $\alpha = 1/\xi$; moments of order $1/\xi$ and above are infinite, so the variance is infinite for $\xi \ge 1/2$ and the mean for $\xi \ge 1$. For $\xi < 0$ the tail is bounded. The lognormal also has $\xi = 0$ in the limit $u \to \infty$, but at any finite threshold it is fitted by a small positive ξ; the fit therefore separates light from heavy tails but cannot by itself separate lognormal from Pareto tails. Both give superpolynomial buffers, so this does not affect our conclusions. We set u at a high empirical quantile of S and test the exponential tail, $\xi = 0$, against a free ξ with a likelihood-ratio test (Methods).

Verdicts. The exponential tail is rejected at 8 of the 10 operating points.

- *Relay-BP, gross code.* Rejected at both points, with the heaviest tails in our data. At $p = 10^{-3}$, $\xi \approx 0.48$ ($\alpha \approx 2.1$), close to the infinite-variance boundary. At $p = 3\times10^{-3}$, $\xi \approx 1.1$–1.5: the fitted tail has an infinite mean, and the survival function in Fig. 1(a) is close to a straight line over four decades until the leg limit. In this regime Eq. (1) cannot hold without a cap, so the leg limit is not optional.

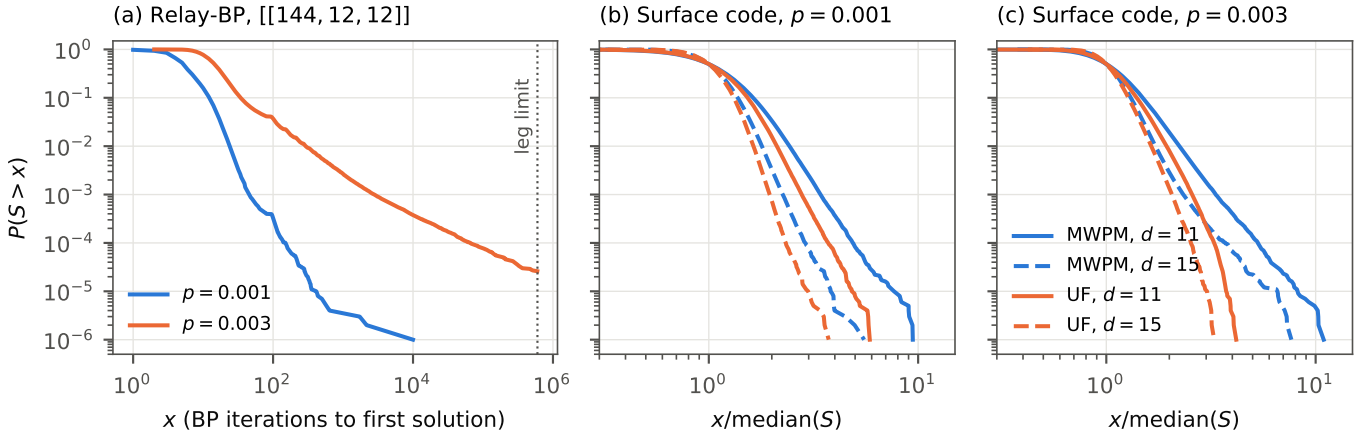


FIG. 1. Empirical survival functions $P(S > x)$ of the decoding work, on log-log axes, where a power law is a straight line. (a) Relay-BP on the [[144, 12, 12]] gross code; S is the number of BP iterations to the first converged leg, and the dotted line marks the leg limit (600 080 iterations). (b), (c) MWPM and UF on the rotated surface code; S is the total count of bounded-cost operations, and x is normalized by the median of S because the two decoders' units differ. Each curve uses 10^6 windows.

- *MWPM, surface code.* Rejected at all four points, with $\xi \approx 0.17$–0.31 ($\alpha \approx 3$–6), also at the higher threshold.
- *UF, surface code.* Rejected at $p = 10^{-3}$ ($\xi \approx 0.15$) but not at $p = 3 \times 10^{-3}$, where the tail is consistent with an exponential. This UF variant is much less accurate than MWPM, with an LER 36 and 210 times higher at $p = 3 \times 10^{-3}$ for $d = 11$ and 15.

We conclude that the decoding work of the decoders in use is typically heavy-tailed. The accurate decoders, Relay-BP on the gross code and MWPM on the surface code, are heavy-tailed at every point we tested, and Relay-BP's tail becomes much heavier with noise strength. The one light tail we found belongs to a fast but inaccurate UF variant. To our knowledge, no previous study has characterised the run-time tail of a BP-family decoder.

VI. DISCUSSION

Parallel-window decoding is necessary for buffer sizing, not only for throughput. PWD was introduced to overcome the throughput limit of sliding windows [5, 6]. With heavy-tailed decoding work it serves a second purpose. A single server, however fast on average, faces the single-big-jump regime of Eq. (9), in which one pathological window determines the overflow probability. Only additional servers raise the number of jumps needed for overflow and thereby change the growth rate in Table I. Sliding-window schemes exist for BP decoders on qLDPC codes [29], but parallel windows do not. Architectures relying on Relay-BP [30] must address this issue by either truncating the decoder or by generalizing PWD to BP-family decoders.

The buffer must support out-of-order release. The multi-server benefit exists only if memory held by a stuck window does not block memory freed by its neighbours. A first-in, first-out ring buffer, the natural structure for a stream, defeats PWD: one slow window pins every later window and restores single-server behaviour. A buffer suited to PWD looks more like a block-pool allocator, with per-window blocks, tracking of window dependencies, and low-latency non-sequential access.

These are memory-system design problems, absent from current decoder benchmarks, which report mean or median latency. The buffer's location is also a technology choice: single-flux-quantum logic decodes fast next to the qubits but lacks dense memory [3, 31, 32], cryogenic CMOS offers SRAM under a tight cooling budget [33, 34], and room-temperature hardware offers ample memory at the cost of bandwidth and latency [10, 11]. Heavy tails favour hierarchical designs in which deep, randomly accessed buffers sit where memory is cheap.

Limitations. With 10^6 shots per point, the fits describe the work distribution down to probabilities of about 10^{-6}; extrapolating to $\varepsilon = 10^{-12}$ assumes that the fitted tail class persists, and the quoted buffer depths are uncertain in their constants. Our conclusions rest on the rare-event assumption $S \gg a$ and on the optimistic D/G/k model of PWD. Window-to-window correlations, fallback decoders and logical operations are outside our scope.

VII. CONCLUSION

In a memory experiment, the backlog problem is the stability condition of the syndrome buffer, and the buffer's required depth is governed by the tail class of the decoding work. Relay-BP on the gross code and MWPM on the surface code are heavy-tailed, so a real-time system must either pay in LER, provision a sliding-window buffer exponential in $\log(1/\varepsilon)$, or run parallel windows with out-of-order release and $O(\log(1/\varepsilon))$ spare decoders.

Syndrome buffering deserves the same architectural attention as decoder throughput, and decoders should be benchmarked by their tails as well as by their medians.

DATA AND CODE AVAILABILITY

Circuit generation, decoder instrumentation patches, data collection and analysis code, and summary results (tail fits and test statistics) are available at `https://github.com/sumeetshirgure/synbuf`. Raw per-shot data are available from the authors on request.

ATTRIBUTION

This research direction spun out of earlier work of ours on low latency MWPM decoders where we were concerned of rare tail events. The initial idea to use queuing theory for modeling syndrome buffer sizes was conceived by S.S. (It is also used to model router packet buffer sizes in classical internetworks.) Claude Opus suggested studying asymptotics of queue sizes with different service time tails. It was also used extensively to research queuing theory and operations research literature, and to run experiments, statistical tests and writing the manuscript. S.N provided guidance and feedback on the manuscript.

METHODS

Circuits and noise

Circuits were generated with Stim [35]. Surface-code memory experiments use Stim's rotated memory circuits with d rounds and uniform circuit-level depolarizing noise of strength p (two-qubit gate depolarization, data-qubit depolarization before each round, and measurement flips). The gross-code circuit uses the syndrome cycle of Bravyi *et al.* [25] with the same noise model and $d = 12$ rounds. Each shot is one d-round window. LERs are per window; for the gross code any logical error among the 12 logical qubits counts as one failure.

Decoders and work metric

All decoders were built from pinned commits with small instrumentation patches; corrections are unchanged. MWPM uses PyMatching's Sparse Blossom [7], and S is the sum of all its bounded-cost operation counters: priority-queue and flooding operations (dual side), alternating-tree, blossom and region operations (primal side), and the final blossom extraction. UF uses the `UnionFindDecoder` of the `ldpc` package [26] in graph (peeling) mode with unweighted growth, and S is the sum of its growth, merge, validity and peeling operation counts. Both surface-code decoders use the same decomposed detector error model and the same shots. Relay-BP [9] uses tuned parameters, 80 pre-iterations, and up to 10^4 legs of up to 60 iterations each (600 080 iterations in total); S is the number of iterations up to and including the first converged leg. Shots that reach the limit without converging (26 shots at $p = 3 \times 10^{-3}$, none at $p = 10^{-3}$) are counted as logical failures and enter the tail fits as right-censored observations.

Thresholds

The threshold u is the empirical $(1 - q)$ quantile of S, so that a fraction q of the windows exceed it. We use $q = 10^{-2}$ for Relay-BP (about 10^4 exceedances), whose bulk is concentrated at a few iterations (median 6 and 15 at the two noise strengths), and $q = 2 \times 10^{-3}$ for MWPM and UF (about 2 000 exceedances). To check that the fit describes the tail rather than the bulk, every fit is repeated at a threshold ten times further out ($q/10$). Work counts are integers; excesses above u receive a continuity correction of $+1/2$.

Likelihood-ratio test

We fit (ξ, σ) by maximum likelihood. The GPD log-likelihood $\mathcal{L}$ of the excesses is maximized over σ at fixed ξ (the profile likelihood) and then over ξ; censored observations contribute the logarithm of the GPD survival function at the censoring point. We test the exponential tail, $H_0 : \xi = 0$, against a free ξ with the statistic

$$\Lambda \;=\; 2\Big[\max_{\xi,\sigma} \mathcal{L}(\xi, \sigma) \;-\; \max_{\sigma} \mathcal{L}(0, \sigma)\Big], \qquad (12)$$

where the null fit is the closed-form exponential maximum-likelihood estimate. Because $\xi = 0$ is an interior point of the parameter space, Λ is asymptotically χ^2_1 under H_0 [36], and we reject H_0 at the 5% level. Inverting the same test gives a 95% profile-likelihood confidence interval for ξ, the set of ξ with $2[\max \mathcal{L} - \max_\sigma \mathcal{L}(\xi, \sigma)] \le 3.84$ [37]. All fitted values, intervals and test statistics are in the code repository.

Buffer estimates

The gross-code buffer depths quoted in the text splice the fitted GPD tail onto the empirical bulk below u, at $\rho = 0.5$. For SWD we solve $\frac{\rho}{1-\rho}\bar{F}_I(x) = \varepsilon$ [Eq. (9)], and for PWD with m spare servers $\bar{F}_I(x)^m = \varepsilon$, the leading order of Ref. [24]. The depth in windows is $B = k + x/a$ with $k = \lfloor \rho \rfloor + m$ decoders and $a = \mathbb{E}[S]/\rho$ (Eq. (6) and its D/G/k analogue); median-decode figures are x divided by the median of S.

[1] B. M. Terhal, Quantum error correction for quantum memories, Reviews of Modern Physics **87**, 307 (2015).

[2] F. Battistel, C. Chamberland, K. Johar, R. W. J. Overwater, F. Sebastiano, L. Skoric, Y. Ueno, and M. Usman, Real-time decoding for fault-tolerant quantum computing: progress, challenges and outlook, Nano Futures **7**, 032003 (2023).

[3] A. Holmes, M. R. Jokar, G. Pasandi, Y. Ding, M. Pedram, and F. T. Chong, Nisq+: Boosting quantum computing power by approximating quantum error correction, in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)* (IEEE, 2020) pp. 556–569.

[4] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, Topological quantum memory, Journal of Mathematical Physics **43**, 4452 (2002).

[5] L. Skoric, D. E. Browne, K. M. Barnes, N. I. Gillespie, and E. T. Campbell, Parallel window decoding enables scalable fault tolerant quantum computation, Nature Communications **14**, 10.1038/s41467-023-42482-1 (2023).

[6] X. Tan, F. Zhang, R. Chao, Y. Shi, and J. Chen, Scalable surface-code decoders with parallelization in time, PRX Quantum **4**, 10.1103/prxquantum.4.040344 (2023).

[7] O. Higgott and C. Gidney, Sparse blossom: correcting a million errors per core second with minimum-weight matching, Quantum **9**, 1600 (2025).

[8] N. Delfosse and N. H. Nickerson, Almost-linear time decoding algorithm for topological codes, Quantum **5**, 595 (2021).

[9] T. Müller, T. Alexander, M. E. Beverland, M. Bühler, B. R. Johnson, T. Maurer, and D. Vandeth, Improved belief propagation is sufficient for real-time decoding of quantum memory (2025), arXiv:2506.01779 [quant-ph].

[10] Google Quantum AI and Collaborators, Quantum error correction below the surface code threshold, Nature **638**, 920 (2024).

[11] B. Barber, K. M. Barnes, T. Bialas, O. Buğdaycı, E. T. Campbell, N. I. Gillespie, K. Johar, R. Rajan, A. W. Richardson, L. Skoric, C. Topal, M. L. Turner, and A. B. Ziad, A real-time, scalable, fast and resource-efficient decoder for a quantum computer, Nature Electronics **8**, 84 (2025).

[12] L. Kleinrock, *Queueing Systems, Volume 1: Theory* (Wiley, New York, 1975).

[13] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action* (Cambridge University Press, 2013).

[14] S. Asmussen, *Applied Probability and Queues*, 2nd ed., Stochastic Modelling and Applied Probability, Vol. 51 (Springer New York, New York, 2003).

[15] D. G. Kendall, Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded markov chain, The Annals of Mathematical Statistics **24**, 338 (1953).

[16] D. V. Lindley, The theory of queues with a single server, Mathematical Proceedings of the Cambridge Philosophical Society **48**, 277 (1952).

[17] R. M. Loynes, The stability of a queue with non-independent inter-arrival and service times, Mathematical Proceedings of the Cambridge Philosophical Society **58**, 497 (1962).

[18] J. F. C. Kingman, The single server queue in heavy traffic, Mathematical Proceedings of the Cambridge Philosophical Society **57**, 902 (1961).

[19] J. S. Sadowsky, The probability of large queue lengths and waiting times in a heterogeneous multiserver queue ii: Positive recurrence and logarithmic limits, Advances in Applied Probability **27**, 567 (1995).

[20] S. Foss, D. Korshunov, and S. Zachary, *An Introduction to Heavy-Tailed and Subexponential Distributions* (Springer New York, 2013).

[21] A. G. Pakes, On the tails of waiting-time distributions, Journal of Applied Probability **12**, 555 (1975).

[22] N. Veraverbeke, Asymptotic behaviour of wiener-hopf factors of a random walk, Stochastic Processes and their Applications **5**, 27 (1977).

[23] P. Embrechts and N. Veraverbeke, Estimates for the probability of ruin with special emphasis on the possibility of large claims, Insurance: Mathematics and Economics **1**, 55 (1982).

[24] S. Foss and D. Korshunov, On large delays in multi-server queues with heavy tails, Mathematics of Operations Research **37**, 201 (2012).

[25] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, High-threshold and low-overhead fault-tolerant quantum memory, Nature **627**, 778 (2024).

[26] J. Roffe, LDPC: Python tools for low density parity check codes, `https://github.com/quantumgizmos/ldpc` (2022).

[27] J. P. III, Statistical inference using extreme order statistics, The Annals of Statistics **3**, 10.1214/aos/1176343003 (1975).

[28] A. A. Balkema and L. de Haan, Residual life time at great age, The Annals of Probability **2**, 10.1214/aop/1176996548 (1974).

[29] A. Gong, S. Cammerer, and J. M. Renes, Toward low-latency iterative decoding of qldpc codes under circuit-level noise (2024), arXiv:2403.18901 [quant-ph].

[30] T. J. Yoder, E. Schoute, P. Rall, E. Pritchett, J. M. Gambetta, A. W. Cross, M. Carroll, and M. E. Beverland, Tour de gross: A modular quantum computer based on bivariate bicycle codes, arXiv preprint arXiv:2506.03094 (2025), arXiv:2506.03094 [quant-ph].

[31] Y. Ueno, M. Kondo, M. Tanaka, Y. Suzuki, and Y. Tabuchi, Qecool: On-line quantum error correction with a superconducting decoder for surface code, in *2021 58th ACM/IEEE Design Automation Conference (DAC)* (IEEE, 2021) pp. 451–456.

[32] I. Byun, J. Kim, D. Min, I. Nagaoka, K. Fukumitsu, I. Ishikawa, T. Tanimoto, M. Tanaka, K. Inoue, and J. Kim, Xqsim: modeling cross-technology control processors for 10+k qubit quantum computers, in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22 (ACM, 2022) pp. 366–382.

[33] B. Patra, R. M. Incandela, J. P. G. van Dijk, H. A. R. Homulle, L. Song, M. Shahmohammadi, R. B. Staszewski, A. Vladimirescu, M. Babaie, F. Sebastiano, and E. Charbon, Cryo-cmos circuits and systems for quantum computing applications, IEEE Journal of Solid-State Circuits **53**, 309 (2018).

[34] S. S. Tannu, Z. A. Myers, P. J. Nair, D. M. Carmean, and M. K. Qureshi, Taming the instruction bandwidth

of quantum computers via hardware-managed error correction, in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO-50 (ACM, 2017) pp. 679–691.

[35] C. Gidney, Stim: a fast stabilizer circuit simulator, Quantum **5**, 497 (2021).

[36] S. S. Wilks, The large-sample distribution of the likelihood ratio for testing composite hypotheses, The Annals of Mathematical Statistics **9**, 60 (1938).

[37] S. Coles, *An Introduction to Statistical Modeling of Extreme Values* (Springer London, 2001).